

Training Course

Helm Chart Introduction



- Deploying app on a running Kubernetes cluster
 - *kubectl* way
 - Helm way
- Helm architecture, Helm 2 vs 3
- ⑩ Learn more about Helm
 - How it works?
 - Charts
 - Template and syntax

Deploying with *kubectl* Way

✧ Many Kubernetes manifest files

- Application deployment and service configuration
- Database deployment and service configuration

✧ Using the Kubernetes client, *kubectl*

- Create Deployment
- Manage Deployment

✧ Weakness

- *kubectl* deployments are not easy to configure, update and rollback
- Requires to track your deployment and modify YAML files (can be error prone)
- Does not allow multiple deployments without updating metadata in manifest files
- Sharing required to have knowledge of deployment configuration and share multiple files

✧ Check out files at: <https://github.com/IBM/questbook/tree/master/v1>

Deploying with *kubect!* Way

- No expertise of Kubernetes deployment needed
- Helm packages all dependencies
- Desired configuration can be passed at runtime as key-value
- Helm tracks deployment making it easy to update and rollback
- Same workload can be deployed multiple times
 - Helm allows assigning workload release names at runtime
- Easy to share

4

What is Helm?

- Helm is a tool that streamlines installation and management of Kubernetes applications
- It uses a packaging format called **charts**
 - A chart is a collection of files that describe Kubernetes resources
- Helm is available for various operating systems like OSX, Linux and Windows
- Run Helm anywhere laptop, CI/C,

6



helm

- *Helm* is the name of the project, the command line client is also named helm.

Tiller

- Tiller is the Helm server. It interacts directly with the Kubernetes API server to install, upgrade, query, and remove Kubernetes resources. It is installed in the Kubernetes cluster.

Chart

- It contains all of the resource definitions necessary to run an application, tool, or service inside of a Kubernetes cluster. A chart is basically a package of pre-configured Kubernetes resources.

Release

- An instance of a chart running in a Kubernetes cluster
- Same chart can be deployed multiple time

Repository

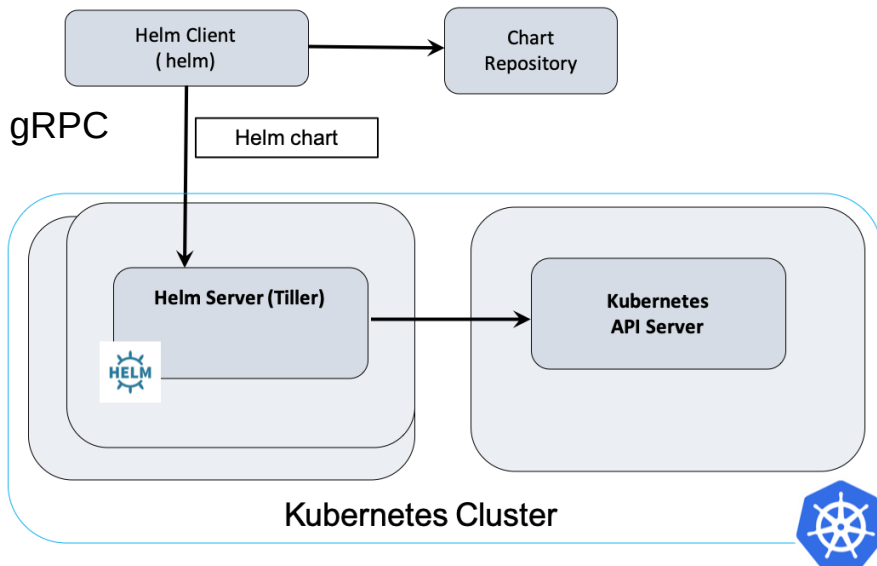
- Place where charts reside and can be shared with others

Helm Chart Introduction

Helm 2 Architecture (Client-Server architecture)

The Helm Client (helm)

- Command-line client for end users
- Interacts with the Tiller server sending charts to be installed/upgrade/uninstall
- Interacts with the chart repository
- The Helm Server (Tiller)
- Interacts with the Helm client and interfaces with the Kubernetes API server
- Renders templates to Kubernetes manifest files
- Install/Upgrade/Uninstall charts into Kubernetes, and tracks the release

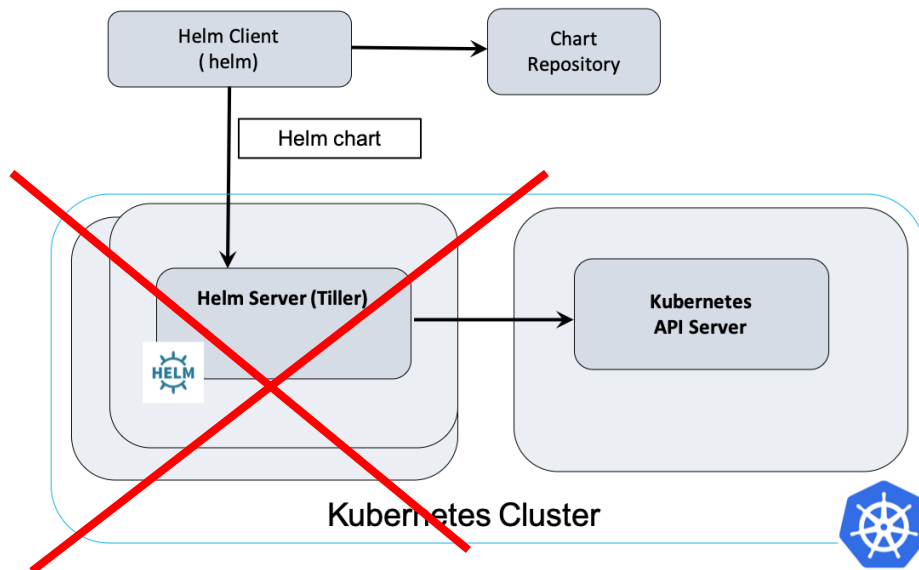


Helm Chart Introduction

Helm 3 appeared

Why helm 3

- Role based access control and Custom resources appeared in k8s
- Provisional dates:
 - Alpha version: Mid 2019
 - Release: 2019 EOY
- main difference

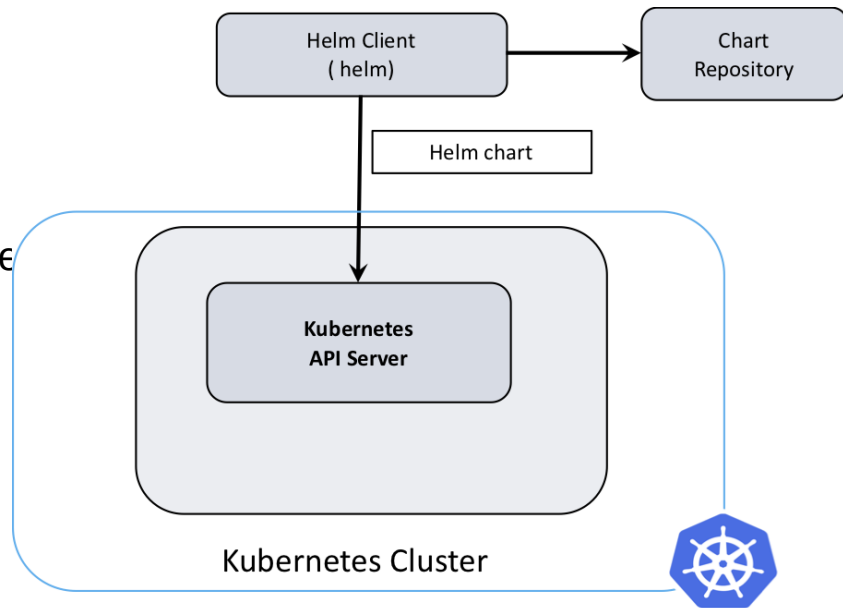


Helm Chart Introduction

Helm 3 appeared

changed ?

- No more Tiller (Server component)
- Three-way strategic merge patch
- Scope for Release Names are limited to Namespace
- Secrets as default storage driver
- JSON Schema Chart Validation
- Consolidation of requirement.yaml into Chart.yaml
- Removal of helm serve
- Chart.yaml apiVersion bump
- CLI Commands Renamed



The Chart File Structure

wordpress/

- ✓ Chart.yaml # A YAML file containing information about the chart
- ✓ LICENSE # OPTIONAL: A plain text file containing the license for the chart
- ✓ README.md # OPTIONAL: A human-readable README file
- ✓ values.yaml # The default configuration values for this chart
- ✓ values.schema.json # OPTIONAL: A JSON Schema for imposing a structure on the values.yaml file
- ✓ charts/ # A directory containing any charts upon which this chart depends.
- ✓ crds/ # Custom Resource Definitions
- ✓ templates/ # A directory of templates that, when combined with values,
will generate valid Kubernetes manifest files.
- ✓ templates/NOTES.txt # OPTIONAL: A plain text file containing short usage notes

Template

```
$ helm create mychart
```

mychart

```
├── Chart.yaml # Information about your chart, me
├── charts # Charts that this chart depends on
├── templates
│   ├── NOTES.txt
│   ├── _helpers.tpl
│   ├── deployment.yaml
│   ├── ingress.yaml
│   ├── service.yaml
│   ├── serviceaccount.yaml
│   └── tests
│       └── test-connection.yaml
└── values.yaml # The default values for your template
```

Template Built-in Objects

- **Release:** This object describes the release itself.
- **Values:** Values passed into the template from the values.yaml file and from user-supplied files.
- **Chart:** The contents of the Chart.yaml file
- **Files:** This provides access to all non-special files in a chart.
- **Capabilities:** This provides information about what capabilities the Kubernetes cluster supports.
- **Template:** Contains information about the current template that is being executed
- how to use object in template: use in action block example: {{
 .Values.attr }}
 helm template _name_

Helm Chart Introduction

Chart

```
apiVersion: The chart API version (required)
name: The name of the chart (required)
version: A SemVer 2 version (required)
kubeVersion: A SemVer range of compatible Kubernetes versions (optional)
description: A single-sentence description of this project (optional)
type: The type of the chart (optional)
keywords:
  - A list of keywords about this project (optional)
home: The URL of this projects home page (optional)
sources:
  - A list of URLs to source code for this project (optional)
dependencies: # A list of the chart requirements (optional)
  - name: The name of the chart (nginx)
    version: The version of the chart ("1.2.3")
    repository: (optional) The repository URL ("https://example.com/charts") or alias ("@repo-name")
    condition: (optional) A yaml path that resolves to a boolean, used for enabling/disabling charts (e.g. subchart1.enabled)
    tags: # (optional)
      - Tags can be used to group charts for enabling/disabling together
    import-values: # (optional)
      - ImportValues holds the mapping of source values to parent key to be imported. Each item can be a string or pair of chart values
    alias: (optional) Alias to be used for the chart. Useful when you have to add the same chart multiple times
maintainers: # (optional)
  - name: The maintainers name (required for each maintainer)
    email: The maintainers email (optional for each maintainer)
    url: A URL for the maintainer (optional for each maintainer)
icon: A URL to an SVG or PNG image to be used as an icon (optional).
appVersion: The version of the app that this contains (optional). Needn't be SemVer. Quotes recommended.
deprecated: Whether this chart is deprecated (optional, boolean)
annotations:
  example: A list of annotations keyed by name (optional).
```

Template syntax

- **Action:** `{{ }}` `{{- }}` work as embedded code
- **pipelines:** `|` `.Values.favorite.food | upper | quote`
- **Function:** Logic and Flow Control Functions, date, ...
- **Flow Control:** if, else, loop, loop with dict, template, block, define
- **Variable:** `{{- $relname := .Release.Name -}}`
- **helper and custom helper:** `{{- include "tpl-example.connection_string" . | nindent 4 }}`
- `_helper.tpl`

```
{{- define "tpl-example.connection_string" }}  
{{- printf "redis://%s:%d/" (include "tpl-example.fullname" .) .Values.port }}  
{{- end }}
```

- **hook:** pre/post with install/upgrade/delete/rollback,.. to do what ever u want like create pod, create service, deployment,..
- **note:** print to console when helm finish
- **test:** use with hook test in tests folder

Chart

- **Dependencies:**
 - define in file chart.yaml and use with subchart value,
 - can work with range version : “=< 8.0.0”
 - can use with repository name or link git “@bitnami/”
 - set condition with value file, or set with **tags** to use with multiple dependencies
 - read value from parent chart with “import-values and export” or “child and parent”
- **Subchart and Global Values:** subchart cannot access parent chart’s values but parent can overwrite subchart’s values
- **Helm package and .ignore**
- **local repo**, repo with **web-server**, use **git**, **oci repo** with index.yaml file
- helm plugin

Thank you

